DP-OPS-0034: System Hardening and Immunological Protocol for Autonomous Agents

1. Executive Summary and Strategic Intent

The operational integrity of the Stela repository depends on the strict enforcement of the Policy of Truth (PoT) and the rigorous management of autonomous actor lifecycles. As the system evolves from a passive repository of code into an active, agentic environment—a transition marked by the recent execution of DP-OPS-0031 and the refactoring of agents R-AGENT-01 through R-AGENT-06—the mechanisms for defining, governing, and verifying agent behavior must mature significantly. The current governance architecture, while functional for static code, exhibits a critical asymmetry: the management of "Skills" (tools) is robust, centralized, and forensically sound, whereas the management of "Agents" (actors) remains distributed, fragmented, and vulnerable to entropic drift.

The existing verification tooling, specifically tools/test/agent.sh, focuses primarily on structural pointer integrity rather than deep behavioral or "immunological" defense. It ensures that files exist but does not validate the *intent* or the *constitutional alignment* of the agents defined within them. This creates a "policy-to-practice" gap [1], where high-level governance documents like PoT.md define the law, but low-level enforcement mechanisms lack the granularity to prevent subtle deviations in agent behavior.

Dispatch Packet OPS-0034 (DP-OPS-0034) is mandated to close this governance gap. The strategic intent is to establish a "System Immune System"—a set of automated, continuously running checks that detect and reject violations of agent constitution before they can manifest as operational failures. This will be achieved through two primary deliverables:

1. **The Agent Promotion Ledger (docs/library/AGENTS.md):** A centralized, immutable record of agent creation, evolution, and retirement, mirroring the proven "Promotion Packet" architecture of docs/library/SKILLS.md.
2. **The Immunological Linter (tools/lint/agent.sh):** A sophisticated enforcement binary that goes beyond syntax checking to validate semantic integrity, registry alignment, and toolchain authorization.

This report provides an exhaustive technical specification for DP-OPS-0034. It synthesizes forensic analysis of the current repository state with advanced research in AI guardrails and agentic registries to propose a governance framework that is both rigorous and scalable. By implementing these controls, the Stela system moves closer to a self-policing architecture capable of sustaining high-velocity agentic work without succumbing to hallucinated authority

or shadow logic.

2. Forensic Analysis of the Current Governance State

A deep forensic review of the repository state, based on the provided artifact dump-full-main-cfbde022d.txt, reveals a distinct maturity gap between the handling of passive tools (Skills) and active actors (Agents). This section analyzes the existing structures to identify the specific friction points that DP-OPS-0034 must resolve.

2.1 The Skill Lifecycle Maturity (The Reference Model)

The "Skills" system, anchored by docs/library/SKILLS.md [2], represents the "Gold Standard" for artifact governance within Stela. Its maturity is defined by several key characteristics that prevent "Shadow IT" (untracked capabilities) and ensure provenance.

2.1.1 Centralized Lifecycle Ledger

The file docs/library/SKILLS.md serves as a "Single Pane of Glass" for the entire skill ecosystem. It is not merely a list; it is a *process document*. It contains the "Promotion Template" that defines exactly what a skill *is*, and it houses the "Candidate Log" and "Promotion Log" that track where a skill *came from*. This centralization ensures that an operator can trace the lineage of any skill (e.g., S-LEARN-06) back to its originating Dispatch Packet (e.g., DP-OPS-0010) and the specific friction context that necessitated its creation.

2.1.2 Explicit Harvest Protocol

The Skills system enforces a "Harvest Engine workflow." The provenance block in every skill file is not manually written; it is generated by ops/lib/scripts/skill.sh harvest. This automation captures critical metadata—Branch, HEAD, Objective, and Friction Context—at the moment of creation. This prevents "provenance hallucination," where an operator or agent might invent a justification for a tool after the fact.

2.1.3 Immunological Guardrails

The skill promotion workflow includes explicit "Drift Preventers." For example, the Promotion Packet template requires the definition of "Stop conditions" and "Anti-hallucination" rules. Furthermore, the tools/lint/library.sh script [2] ensures that every skill file on disk has a corresponding entry in the registry (docs/ops/registry/SKILLS.md), preventing "ghost skills" from existing in the codebase.

2.2 The Agent Lifecycle Asymmetry (The Friction Point)

In stark contrast, the "Agent" system currently lacks this level of rigorous centralization. The defining artifacts for agents are distributed and disjointed, creating a governance surface that is difficult to audit and prone to drift.

2.2.1 Distributed Definition Risks

Currently, an agent's identity is split across multiple files:

- **The Registry (docs/ops/registry/AGENTS.md):** Contains a simple table mapping IDs (e.g., R-AGENT-01) to Names and Provenance.[2]
- **The Constitution (docs/library/agents/R-AGENT-XX.md):** Contains the actual role definition, pointers, and toolchain authorization.[2]
- **The Tooling (ops/lib/scripts/agent.sh):** Handles the mechanics of creation but currently writes directly to the registry and the SoP, bypassing a centralized history ledger.[2]

This distribution means there is no single file analogous to SKILLS.md where an operator can view the *template* for a new agent or the *history* of agent evolution. If an operator wants to create a new agent, they must currently look at an existing agent file and copy its structure, a practice that encourages "copy-paste drift" where errors or legacy patterns are propagated to new agents.

2.2.2 The "Ghost Agent" Vulnerability

The current verification script, tools/test/agent.sh, performs only superficial checks. It verifies that ## Pointers and ## Scope Boundary sections exist, and it checks that toolchain binaries are present.[2] However, it does not enforce a strict 1:1 mapping between the registry and the library. It is theoretically possible for a "Ghost Agent" (e.g., R-AGENT-99.md) to exist in the docs/library/agents/ directory without being registered in docs/ops/registry/AGENTS.md. Such an agent could be invoked by a confused LLM or a malicious actor, operating outside the visibility of the primary governance logs.

2.2.3 Lack of Semantic Enforcement

While DP-OPS-0031 introduced "Pointer-First" constitutions, the current system relies on the *agent's own compliance* to respect those pointers. There is no external linter that validates, for example, that an agent is not referencing a deleted documentation file or that it is not attempting to load the agent library recursively (a "Context Hazard"). The system relies on *trust* rather than *verification*, a violation of the zero-trust principles outlined in PoT.md.

3. Theoretical Framework: The Immune System for AI

To design an effective hardening protocol, we must look beyond simple software testing and adopt concepts from immunological defense and "Agentic AI" governance. The core challenge in governing autonomous agents is their non-deterministic nature; unlike static code, agents act probabilistically and can adapt to their environment.[3] Therefore, governance must be dynamic and "context-aware" rather than static.

3.1 The Policy-to-Practice Gap

Research by Wang et al. (2025) and others identifies a critical "policy-to-practice" gap in AI governance.[1] High-level policies (like PoT.md) are human-readable but not machine-enforceable. Low-level rules (like code linters) are machine-enforceable but lack semantic context. To bridge this, we need "Policy as Prompt" or "Guardrail Linting"—mechanisms that translate high-level intent into executable constraints.

In the Stela context, tools/lint/agent.sh will serve as this bridge. It will not just check for syntax errors; it will check for *constitutional violations*. For example, PoT.md mandates "No contractions" and "Absolute literalism." The linter can enforce the former via regex and the latter via structural constraints (e.g., requiring explicit "Stop Conditions" in every agent definition).

3.2 Agent Registries as Governance Anchors

The concept of an "AI Agent Registry" is central to modern agentic architecture. As noted in industry analyses [5], a registry acts as a "phone book" or discovery platform that enables interoperability and governance. However, unlike complex enterprise registries involving distributed hash tables or blockchain proofs, Stela's "Flat Truth" philosophy requires a local, file-based registry.

The Stela registry system (docs/ops/registry/AGENTS.md) combined with the proposed ledger (docs/library/AGENTS.md) creates a "Double-Entry Bookkeeping" system for governance. The Registry tracks *current state* (who exists now), while the Ledger tracks *lifecycle events* (who was created, why, and by whom). This dual structure is essential for forensic auditability—if an agent behaves anomalously, the Ledger allows us to rewind the tape and see the exact objectives and constraints under which it was born.

3.3 Immunological Defense Layers

Effective AI guardrails operate at multiple layers: Input, Output, and Tooling.[7] The proposed tools/lint/agent.sh script will implement checks across these layers:

- **Input Guardrails:** Ensuring the agent's "Pointers" (its input context) are valid and do not include forbidden knowledge (Context Hazards).
- **Tooling Guardrails:** validating the "Authorized Toolchain" to ensure agents only have access to approved binaries (ops/bin/*).
- **Output Guardrails:** Enforcing the presence of "Scope Boundaries" and "Drift Preventers" in the agent's definition, constraining its potential output actions.

4. Architectural Specification: The Agent Promotion Ledger (AGENTS.md)

The first deliverable of DP-OPS-0034 is the creation of docs/library/AGENTS.md. This file is not a passive document; it is an active governance surface. It must mirror the functionality of

SKILLS.md but be adapted for the unique properties of agents.

4.1 Schema and Structure

The file will reside in docs/library/ and adhere to the following strict schema:

4.1.1 Header and Warning

The file must begin with a clear declaration of its purpose and a "Context Hazard" warning. Agents reading this file should understand they are observing the *governance layer*, not the *operational layer*.

Agent Promotion Ledger (Governance)

> **CONTEXT HAZARD:** This file governs the lifecycle of agents. It is for Operator and Governance use only.
>
> Active agents should not load this file unless explicitly tasked with governance auditing.

4.1.2 The Harvest Workflow Definition

Just as SKILLS.md defines the "Harvest Engine workflow," AGENTS.md must define the "Agent Birth Protocol." This section will mandate the use of ops/lib/scripts/agent.sh harvest for creating new agents. It establishes the rule that *no agent is born by manual file creation*; every agent must have a machine-generated provenance block.

4.1.3 The Promotion Packet Template

This is the core of the file. The Promotion Packet is a structured metadata block that must be completed before an agent can be promoted from "Draft" to "Canon." It acts as a "Birth Certificate" and "Job Description" combined.

Template Fields:

- **Candidate Name & ID:** The unique identifier (e.g., R-AGENT-07) and human-readable name.
- **Role Summary:** A one-sentence definition of the agent's purpose (e.g., "Forensic auditor of system logs").
- **Specialization:** The specific domain expertise required (e.g., "Pattern matching, anomaly detection").
- **Provenance:** Data captured automatically by the harvest tool (DP-ID, Branch, HEAD, Objective, Friction Context).
- **Drift Preventers:** A critical section requiring explicit definition of:
  - *Scope Boundary:* Where the agent is allowed to act (e.g., "Read-only access to logs/").

    - *Anti-Hallucination:* Rules for verifying inputs (e.g., "Must verify file existence before reading").
    - *Negative Checks:* What the agent must *not* do (e.g., "Do not attempt to push to main").
- **Definition of Done:** The checklist for promotion (Registry update, Index update, SoP entry).
- **Verification:** The list of linter commands that must pass (including the new tools/lint/agent.sh).

4.1.4 The Candidate and Promotion Logs

The file will conclude with two append-only log sections:

- **Candidate Log:** Tracks drafts that are currently in storage/handoff/ or storage/dp/intake/. This provides visibility into "agents in utero."
- **Promotion Log:** Tracks the finalization of agents. This serves as the permanent historical record of when an agent joined the workforce and under which DP authority.

4.2 Comparative Analysis: Skills vs. Agents

It is crucial to understand why AGENTS.md mirrors SKILLS.md yet remains distinct.

Feature	SKILLS.md	AGENTS.md (Proposed)	Rationale
Primary Entity	Reusable Workflow (Tool)	Autonomous Actor (Persona)	Agents execute Skills; they are distinct governance objects.
Provenance	Capture "Lesson Learned"	Capture "Operational Need"	Skills come from *experience*; Agents come from *requirements*.
Risk Profile	Low (Passive execution)	High (Active decision making)	Agents can decide to act; Skills wait to be used.
Linter Focus	Context Hazard (Don't pollute context)	Immunological (Don't hallucinate authority)	Agents need stricter "Scope Boundary"

			enforcement.
Registry	docs/ops/registry/SKILLS.md	docs/ops/registry/AGENTS.md	Separate registries for separate entity types.

5. Architectural Specification: The Immunological Linter (agent.sh)

The second deliverable is tools/lint/agent.sh. This script is the enforcement arm of the new governance protocol. It translates the text of AGENTS.md into blocking CI/CD gates.

5.1 Logic Levels and Heuristics

The linter will operate using a "Defense in Depth" strategy, applying five distinct levels of verification.

Level 1: Constitutional Integrity (The File)

The script will parse every file in docs/library/agents/ to ensure structural compliance.

- **Header Check:** Does the file start with # Agent: <name>?
- **Section Check:** Are ## Provenance, ## Role, ## Specialization, ## Pointers, and ## Scope Boundary present?
- *Rationale:* A file missing these sections is not an agent; it is a rogue text file masquerading as one.

Level 2: Registry Alignment (The Ledger)

The script will perform a bi-directional sync check between the files and the registry (docs/ops/registry/AGENTS.md).

- **Ghost Check:** If R-AGENT-07.md exists but is not in the registry, it is a "Ghost Agent" (Shadow IT) and must be flagged.
- **Dead End Check:** If the registry lists R-AGENT-07 but the file is missing, it is a "Dead End" (Broken Pointer) and must be flagged.
- *Rationale:* Discrepancies here indicate a failure in the promotion workflow or a manual bypass of the harvest/promote script.

Level 3: Pointer Validity (The Reality Check)

The script will parse the ## Pointers section of each agent. For every file referenced (e.g., -PoT.md), it will perform a test -f check.

- *Rationale:* Agents often "hallucinate" authoritative documents that do not exist or have

been moved. This check prevents agents from grounding their logic in phantom documentation.

Level 4: Context Hazard Defense

The script will grep the agent file for references to docs/library/agents/.

- *Rationale:* An agent definition should never load the entire agent library. This creates a recursive loop (an agent loading itself loading itself) and pollutes the context window. Agents should define their role, not read the roles of everyone else (unless specifically authorized).

Level 5: Toolchain Authorization

The script will parse the "Authorized toolchain" list. It will verify that every listed tool (e.g., ops/bin/dump) exists and is executable.

- *Rationale:* If a tool is deprecated or moved, the agent's capability breaks. This check serves as a "dependency lock" for agent capabilities.

5.2 Implementation Logic (Bash)

The script will be written in robust Bash (set -euo pipefail) to ensure reliability. It will utilize standard Unix utilities (grep, awk, sed, test) to maintain portability and avoid dependencies on external runtimes like Python or Node.js, adhering to the repository's "Ops Lane" tone.[2]

6. Detailed Implementation Plan: DP-OPS-0034

This section defines the exact steps and content required to execute the Dispatch Packet.

6.1 Deliverable 1: docs/library/AGENTS.md

The file will be created with the following content structure. The template matches the rigorous requirements of the "Promotion Packet" discussed in Section 4.1.3.

Content Preview (Excerpt):

Agent Promotion Ledger

> **CONTEXT HAZARD:** Governance surface. Not for payload execution.

Harvest Engine Workflow

Use ops/lib/scripts/agent.sh harvest to generate candidates.

Promotion Packet Template

Promotion Packet: R-AGENT-XX - [NAME]

- Candidate name: [NAME]
- Proposed Agent ID: R-AGENT-XX
- Role:
- Provenance:
- Drift Preventers:
  - Scope Boundary:
  - Anti-hallucination: Verify inputs.
- Verification:
  - tools/lint/agent.sh
    ...

6.2 Deliverable 2: tools/lint/agent.sh

The script will be created at tools/lint/agent.sh. It will include the logic defined in Section 5.1.

Key Script Logic (Pseudocode):

1. **Setup:** Define paths (AGENTS_DIR, REGISTRY_FILE, LEDGER_FILE).
2. **Ledger Check:** Verify AGENTS.md exists and contains mandatory headers.
3. **Registry Sync:**
   - Load IDs from Registry.
   - Load IDs from File System.
   - Compare and report Ghosts/Dead Ends.
4. **Agent Scan (Loop):**
   - For each agent file:
     - Check for mandatory sections (grep -q).
     - Extract Pointers and verify (test -f).
     - Extract Toolchain and verify (test -x).
     - Check for Context Hazards (grep "docs/library/agents").
5. **Reporting:** Output PASS/FAIL status.

6.3 Update to ops/lib/scripts/agent.sh

The existing ops/lib/scripts/agent.sh must be updated to reference the new ledger. Currently, it appends to SoP.md directly. It must now:

1. Append new candidates to the "Candidate Log" in AGENTS.md.
2. Append completed promotions to the "Promotion Log" in AGENTS.md.
3. Ensure the "Promotion Packet" it generates matches the new schema in AGENTS.md.

7. Operational Impact and Governance Scenarios

The implementation of DP-OPS-0034 fundamentally changes the "physics" of the Stela repository.

7.1 Scenario: The "Shadow Agent" Attempt

- *Before:* An operator manually creates docs/library/agents/R-AGENT-99.md to perform a quick, unauthorized task. They do not register it.
- *After:* The repo-gates workflow runs tools/lint/agent.sh. The linter detects R-AGENT-99.md on disk but finds no entry in docs/ops/registry/AGENTS.md. It flags a "Ghost Agent" violation and fails the build. The PR cannot be merged.

7.2 Scenario: The "Hallucinating Architect"

- *Before:* R-AGENT-01 (Architect) is defined with a pointer to docs/architecture/legacy-diagram.md. This file was deleted months ago. The agent hallucinates the content of the missing file during execution.
- *After:* tools/lint/agent.sh runs Level 3 (Pointer Validity). It attempts to test -f docs/architecture/legacy-diagram.md, fails, and reports a "Broken Pointer." The agent definition must be updated to remove the dead link before it can pass the gate.

7.3 Scenario: The "Toolchain Drift"

- *Before:* R-AGENT-06 (Security) relies on a binary ops/bin/scan-secrets. A refactor moves this binary to tools/sec/scan. The agent breaks in production.
- *After:* tools/lint/agent.sh runs Level 5 (Toolchain Auth). It checks for ops/bin/scan-secrets, sees it is missing, and fails the build. The operator is forced to update the agent's toolchain definition to match the new reality.

8. Dispatch Packet OPS-0034 Construction

The following text constitutes the full Dispatch Packet to be entered into TASK.md.

DP-OPS-0034: Agent System Hardening (Immune System)

3.1 Freshness Gate (Must Pass Before Work)

- **Base Branch:** main
- **Required Work Branch:** work/agent-hardening-0034
- **Base HEAD:** cfbde022d

3.2 Required Context Load

- **Loaded:** PoT, SoP, CONTEXT, MAP
- **Job-Specific:** ops/lib/scripts/agent.sh, docs/library/SKILLS.md (reference)

3.3 Scope and Safety

- **Objective:** Harden the agent system by establishing a centralized Promotion Ledger (AGENTS.md) and a rigorous Immunological Linter (agent.sh).
- **Non-Goals:** Refactoring the internal logic of existing agents (unless required for linter compliance).
- **Target Files allowlist (hard gate):**
  - docs/library/AGENTS.md (new)
  - tools/lint/agent.sh (new)
  - ops/lib/scripts/agent.sh
  - docs/library/INDEX.md
  - SoP.md
  - TASK.md

3.4 Execution Plan (A-E Canon)

3.4.1 State

- **Context:** The agent system lacks a centralized promotion ledger and rigorous behavioral linting.
- **Drift:** Agent definitions are scattered; verification is structural only, not semantic.
- **Desired State:** A unified AGENTS.md ledger and a strict agent.sh linter enforcing constitutional integrity.

3.4.2 Request

1. **Create docs/library/AGENTS.md:** Mirror the structure of SKILLS.md. Include "Harvest Engine Workflow," "Promotion Packet Template," "Candidate Log," and "Promotion Log." Ensure the template enforces "Drift Preventers" and "Definition of Done."
2. **Create tools/lint/agent.sh:** Implement the 5-level immunological check:
   - Level 1: Constitutional Integrity (Headers, Sections).
   - Level 2: Registry Alignment (Ghosts, Dead Ends).
   - Level 3: Pointer Validity (Reality Check).
   - Level 4: Context Hazard Defense (Recursive loading).
   - Level 5: Toolchain Authorization (Binary check).
3. **Update ops/lib/scripts/agent.sh:** Refactor the harvest/promote logic to write to the new AGENTS.md logs instead of just the SoP.
4. **Register Surfaces:** Add docs/library/AGENTS.md to docs/library/INDEX.md.
5. **Verify:** Run the new linter against existing agents. Fix any immediate violations.

3.4.3 Changelog

- Created docs/library/AGENTS.md.
- Created tools/lint/agent.sh.
- Updated ops/lib/scripts/agent.sh for ledger integration.
- Updated docs/library/INDEX.md.

3.4.4 Patch / Diff

- Standard unified diff.

3.4.5 Receipt (Required)

- **Verification:** ./ops/bin/dump --scope=platform
- **Linter Check:** bash tools/lint/agent.sh
- **Proof Bundle:** storage/handoff/DP-OPS-0034-RESULTS.md

4 Closeout (Mandatory)

- [ ] Produce/Update: storage/handoff/DP-OPS-0034-RESULTS.md
- [ ] Update SoP.md.
- [ ] Run ops/bin/prune.
- [ ] Run ops/bin/llms.

9. Conclusion

DP-OPS-0034 marks a critical inflection point in the Stela governance maturity model. By moving from "trust-based" agent definition to "verification-based" immunological defense, the system inoculates itself against the entropy that plagues autonomous architectures. The introduction of AGENTS.md provides the necessary historical and procedural grounding, while tools/lint/agent.sh ensures that this grounding is respected in every commit. This effectively closes the "policy-to-practice" gap, transforming the high-level mandates of the Policy of Truth into executable, non-negotiable code. The result is a repository that is not just a storage space for code, but a self-governing environment for intelligent actors.

Data Tables and Structural Reference

Table 1: Comparative Lifecycle Maturity (Pre vs. Post DP-OPS-0034)

Feature	Skills System (Baseline)	Agent System (Current)	Agent System (Target State)
Ledger File	docs/library/SKILLS.md	*None*	docs/library/AGENTS.md
Registry	docs/ops/registry/SKILLS.md	docs/ops/registry/AGENTS.md	docs/ops/registry/AGENTS.md
Harvest Tooling	skill.sh harvest	agent.sh harvest	agent.sh harvest

		(partial)	(integrated)
Promotion Packet	Mandatory (Template in SKILLS.md)	Ad-hoc / Implicit	Mandatory (Template in AGENTS.md)
Linter Enforcement	lint/library.sh (Existence)	test/agent.sh (Basic)	lint/agent.sh (Deep Semantic)
Drift Prevention	Explicit "Drift Preventers" section	Loose prose	Explicit "Drift Preventers" section

Table 2: Immunological Linter Logic Levels

Check Level	Name	Description	Failure Condition
Level 1	Constitutional Integrity	Validates file structure and mandatory sections.	Missing ## Provenance, ## Role, or ## Scope Boundary.
Level 2	Registry Alignment	Ensures 1:1 mapping between files and registry.	"Ghost" agent (file with no registry entry) or "Dead End" (registry entry with no file).
Level 3	Pointer Validity	Verifies all referenced docs exist.	## Pointers references a non-existent file (e.g., deleted doc).
Level 4	Context Hazard	Prevents recursive loading.	Agent definition referencing docs/library/agents / directly.
Level 5	Toolchain Auth	Verifies tool	"Authorized toolchain"

		availability.	references a missing or non-executable binary.

Works cited

1. The AI Agent Code of Conduct: Automated Guardrail Policy-as-Prompt Synthesis - arXiv, accessed February 7, 2026, https://arxiv.org/html/2509.23994v1
2. dump-full-main-cfbde022d.txt
3. AI Agent Evaluation: Frameworks, Strategies, and Best Practices | by Dave Davies - Medium, accessed February 7, 2026, https://medium.com/online-inference/ai-agent-evaluation-frameworks-strategies-and-best-practices-9dc3cfdf9890
4. Different Evals for Agentic AI: Methods, Metrics & Best Practices - testRigor AI-Based Automated Testing Tool, accessed February 7, 2026, https://testrigor.com/blog/different-evals-for-agentic-ai/
5. What is AI Agent Registry - A Complete Guide - TrueFoundry, accessed February 7, 2026, https://www.truefoundry.com/blog/ai-agent-registry
6. Evolution of AI Agent Registry Solutions: Centralized, Enterprise, and Distributed Approaches - arXiv, accessed February 7, 2026, https://arxiv.org/html/2508.03095v3
7. Securing AI Agents with Layered Guardrails and Risk Taxonomy - Enkrypt AI, accessed February 7, 2026, https://www.enkryptai.com/blog/securing-ai-agents-a-comprehensive-framework-for-agent-guardrails